Dynamic Platoon Formation of Multi-Type Autonomous Vehicles for Sustainable Urban Mobility

***Abstract*—This paper addresses the energy inefficiency of single-occupancy vehicles by introducing a novel cooperative autonomous vehicle system where smaller Passive Vehicles (PVs) can be physically towed by larger Active Vehicles (AVs) in a shared road segment. Unlike traditional platooning that relies solely on aerodynamic drafting, our approach enables PV propulsion energy to be fully offset by the towing AV during the shared segment travel, though the AV bears the additional load. We formulate the platoon formation problem as a constrained optimization problem and propose two algorithms: a Greedy Maximum-Weight Matching algorithm and an Iterative Linear Assignment (ILA) algorithm using the Jonker–Volgenant method. A key innovation is the multi-segment matching capability, allowing a single PV to be towed by multiple AVs across different route segments. Experimental evaluation on synthetic scenarios with up to 400 PVs and 80 AVs demonstrates that both algorithms achieve 25–52% distance coverage (proportional to energy savings under constant per-distance propulsion) depending on capacity. Our experimental evaluation shows that the ILA consistently matches or exceeds greedy performance across all tested scenarios, achieving 0.2–0.9 percentage-point higher saving rates with 2.8–11.5× lower runtime. The complete codebase is available at https://github.com/JayReeMarine/Integrating-Multi-Type-Autonomous-Vehicles-for-Sustainable-Urban-Mobility.**

***Index Terms*—autonomous vehicles, vehicle platooning, energy efficiency, linear assignment, Jonker–Volgenant algorithm, combinatorial optimization, intelligent transportation systems**

I. Introduction

Single-occupancy travel remains a major barrier to sustainable urban mobility. U.S. National Household Travel Survey data show that drive-alone commuting dominates trip demand [1], while the transportation sector contributes about 29% of U.S. greenhouse gas emissions [2]. This inefficiency is pronounced when many vehicles traverse overlapping routes but still consume propulsion energy independently. With rapid advances in autonomous driving [3], future transportation can realize physically coordinated vehicle platoons that go beyond independent driving.

In a future where vehicles are fully autonomous, physical coordination becomes feasible: a large AV could mechanically couple with smaller vehicles on a shared highway segment, towing them and offsetting a substantial share of their independent propulsion energy for that portion of the trip [3], [13].

To address this, we introduce a new problem that we call *dynamic platoon formation*. We consider two classes of autonomous vehicles: larger *Active Vehicles (AVs)* capable of physically towing smaller vehicles, and smaller *Passive Vehicles (PVs)* that can be attached to AVs during shared road segments. Unlike drafting-based platooning, which yields modest aerodynamic savings while vehicles remain physically separated, our model assumes mechanical coupling: when a PV is attached to an AV, the PV's own propulsion on that segment is *fully offset*, with the towing load borne by the AV. Given a set of AVs and PVs with known, fixed routes (AVs do not deviate from their pre-planned paths), entry times, and speeds, the goal is to **maximize the total PV distance covered by towing**. We use covered distance as a proxy for PV-side propulsion-energy reduction, and reserve physics-level load modeling for future work.

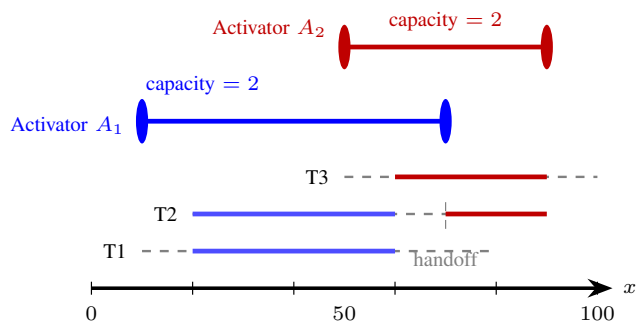


Fig. 1: Conceptual overview of towing-based cooperation. Dashed gray segments denote self-driving portions (PV uses own propulsion), and solid colored segments denote towed travel (PV propulsion offset by AV). Activators A_1 and A_2 each have capacity 2; $T2$ is towed by A_1 to $x = 60$, self-drives to $x = 70$, then is handed off to A_2.

Figure 1 illustrates a concrete example. Three PVs ($T1$, $T2$, $T3$) travel along a shared highway, and two AVs (A_1, A_2) each have towing capacity 2. In the depicted solution, $T1$ is towed by A_1 over $[20, 60]$; $T2$ is towed by A_1 over $[20, 60]$, then travels on its own over $[60, 70]$, and is handed off to A_2 over $[70, 90]$, demonstrating *multi-segment matching*—a single PV served by multiple AVs across different route segments; and $T3$ is towed by A_2 over $[60, 90]$. Between $x = 20$ and $x = 60$, A_1 simultaneously tows $T1$ and $T2$, reaching its capacity of 2. Solid segments denote towed (propulsion-free) travel; dashed segments denote self-driving.

Prior work spans platooning [4], [5], ridesharing and routing [6]–[8], and assignment optimization [9], [10]. However, none of these lines of work models physical towing that combines multi-segment PV coverage with point-wise AV

capacity constraints—i.e., the number of PVs an AV can tow simultaneously varies along its route as vehicles attach and detach—in a single framework. Our formulation targets that gap.

We formulate this as an optimization problem: find segment assignments that maximize total towed PV distance, subject to spatial overlap, AV capacity, and temporal synchronization constraints. This multi-segment matching problem generalizes the generalized assignment problem, which is known to be NP-hard [11], so exact solutions are intractable for realistic fleet sizes. We propose two approximate solution strategies. First, a *Greedy Maximum-Weight Matching* algorithm iteratively selects the highest-benefit feasible AV–PV towing pair and commits to it, repeating until no feasible pairs remain. This approach is fast but myopic. Second, an *Iterative Linear Assignment (ILA)* algorithm decomposes the multi-segment problem into a sequence of bipartite matching subproblems: at each iteration, it constructs a cost matrix that encodes the towing benefit of every feasible AV–PV pair, expands each AV into virtual slots to encode capacity, solves the resulting Linear Sum Assignment Problem (LSAP) optimally via the Jonker–Volgenant method [12], updates PV states to reflect newly covered segments, and repeats until no further feasible assignments exist. For tractability, we evaluate both algorithms under a one-dimensional highway abstraction.

This paper makes three main contributions. First, we formalize the dynamic platoon formation problem with multi-segment matching and point-wise AV capacity constraints. For tractability, we instantiate and evaluate this formulation on a one-dimensional highway abstraction. Second, we develop and compare two approximate solution strategies: a greedy matching algorithm and an Iterative Linear Assignment (ILA) method with virtual-slot capacity encoding. Third, we provide a controlled experimental study (50–80 AVs, 200–400 PVs) showing substantial coverage savings, with ILA consistently matching or exceeding greedy performance across all tested scenarios (0.2–0.9 percentage-point higher saving rates, 2.8–11.5× faster).

II. Related Work

A. Platooning and Drafting

Platooning studies report 10–20% fuel savings primarily through aerodynamic drafting [4], [5], [13]. Vehicles remain physically separated, coordinating spacing and speed via cooperative adaptive cruise control [14]. More recent work has examined platoon coordination on highway networks, including scheduling when and where vehicles should form platoons to minimize fuel consumption [15]. Our setting differs fundamentally: we model physical towing rather than aerodynamic drafting. When a PV is attached, its propulsion is fully offset by the towing AV, and the matching decisions involve dynamic attach/detach operations under point-wise capacity constraints—a problem structure absent from standard drafting formulations.

B. Ridesharing and Vehicle Coordination

Ridesharing and dial-a-ride methods optimize passenger-to-vehicle assignment with timing and capacity constraints [6]–[8], [16]. Dynamic trip-vehicle assignment approaches [16] and multi-agent intersection coordination [17] address related resource-allocation problems in transportation. However, in our problem the decision variable is vehicle-to-vehicle coupling over road segments, not passenger placement. Capacity is enforced point-wise along each AV's path (the set of attached PVs changes at every coupling/decoupling point), which differs from conventional seat-capacity or intersection-slot constraints.

III. Problem Formulation

For ease of understanding, we formalize the platoon formation problem on a simplified one-dimensional highway model. This abstraction captures the essential optimization trade-offs while enabling tractable analysis. Extension to general road networks is discussed in Section VI-A.

A. Notation Summary

Table I summarizes the key notation used throughout this paper.

TABLE I: Summary of Notation

Symbol	Description
L	Highway length
$\mathcal{A}$	Set of Active Vehicles (AVs), $\lvert\mathcal{A}\rvert = N$
$\mathcal{P}$	Set of Passive Vehicles (PVs), $\lvert\mathcal{P}\rvert = M$
e_i^a, x_i^a	Entry and exit points of AV a_i
e_j^p, x_j^p	Entry and exit points of PV p_j
C_i	Towing capacity of AV a_i
t_i^a, v_i^a	Entry time and speed of AV a_i
t_j^p, v_j^p	Entry time and speed of PV p_j
$\ell_j, \hat{t}_j$	Current position and arrival time of PV p_j
d_{ij}	Shared travel distance between a_i and p_j
S_{ij}	Energy saving when p_j is towed by a_i
cp_{ij}, dp_{ij}	Coupling and decoupling points
L_{min}	Minimum shared distance for platooning
τ	Time tolerance for coupling synchronization

B. System Model

We consider a coordinated platoon formation system operating on a unidirectional highway segment of length L. The system comprises two distinct vehicle classes.

Definition 1 (Active Vehicle)**.** *An Active Vehicle (AV) $a_i \in \mathcal{A}$ is a larger autonomous vehicle capable of towing multiple smaller vehicles. Each AV is characterized by the tuple $(e_i^a, x_i^a, C_i, t_i^a, v_i^a)$ where $e_i^a, x_i^a \in [0, L]$ are entry and exit points, $C_i \in \mathbb{Z}^+$ is the towing capacity, and t_i^a, v_i^a are the entry time and constant speed.*

Definition 2 (Passive Vehicle)**.** *A Passive Vehicle (PV) $p_j \in \mathcal{P}$ is a smaller autonomous vehicle that can be towed by AVs. Each PV is characterized by the tuple $(e_j^p, x_j^p, t_j^p, v_j^p)$ where $e_j^p, x_j^p \in [0, L]$ are entry and exit points, and t_j^p, v_j^p are entry time and self-driving speed.*

C. Shared Path and Energy Model

For an AV a_i and PV p_j, the *shared path* is the overlap of their routes:

$$d_{ij} = \max(0, \min(x_i^a, x_j^p) - \max(e_i^a, e_j^p)) \tag{1}$$

This quantity is the physically towable overlap between the two routes. It is needed to define both feasibility and the towing benefit of a candidate segment.

The coupling point cp_{ij} (where the PV attaches) and decoupling point dp_{ij} (where the PV detaches) are defined as $cp_{ij} = \max(e_i^a, e_j^p)$ and $dp_{ij} = \min(x_i^a, x_j^p)$, respectively.

The energy saving when PV p_j is towed by AV a_i is proportional to shared distance, i.e., $S_{ij} = \alpha d_{ij}$, where α is an energy coefficient. We set $\alpha = 1$, so saving equals covered distance.

D. Multi-Segment Matching

A key idea of our formulation is *multi-segment matching*: a single PV can be matched to multiple AVs across different segments of its route.

Definition 3 (Segment Assignment)**.** *A segment assignment for PV p_j to AV a_i is a tuple (p_j, a_i, s, e) where $[s, e] \subseteq [e_j^p, x_j^p]$ is the segment during which p_j is towed by a_i.*

For PV p_j, its route from e_j^p to x_j^p can be partitioned into covered segments (towed by AVs, thus saving propulsion energy) and uncovered segments (self-driven, with no towing-based saving). This is necessary because one AV often covers only part of a PV route, so multiple feasible handoffs may be required to increase total coverage.

E. Optimization Problem

Let $\mathcal{X}$ denote the set of all segment assignments (p_j, a_i, s, e) where PV p_j is towed by AV a_i over segment $[s, e]$. The decision variables are the selected segment assignments in $\mathcal{X}$. The optimization problem is:

$$\max \quad \sum_{(p_j, a_i, s, e) \in \mathcal{X}} (e - s) \tag{2}$$

$$\text{s.t.} \quad |\{(p_j, a_i, s, e) \in \mathcal{X} : x \in [s, e]\}| \le C_i \quad \forall i \in [N], \forall x \in [e_i^a, x_i^a] \tag{3}$$

$$\text{Segments for each } p_j \text{ are pairwise disjoint} \tag{4}$$

$$(e - s) \ge L_{min} \quad \forall (p_j, a_i, s, e) \in \mathcal{X} \tag{5}$$

The objective (2) maximizes total covered PV distance, which is our proxy for PV propulsion-energy reduction.

Constraint Interpretation:

- **Point-wise Capacity** (3): At any position x along AV a_i's route, the number of simultaneously towed PVs cannot exceed C_i. This is enforced point-wise (not only per AV globally) because attachments change along the route as PVs couple and decouple.
- **Non-overlap** (4): A PV cannot be towed by two AVs simultaneously at any position.
- **Minimum length** (5): Each towing segment must exceed L_{min} to justify coupling overhead.

Constraint (3) applies at every position x, but the simultaneous tow count is a step function that changes only at coupling/decoupling endpoints. It therefore suffices to check capacity only at these *critical points*; the following lemma formalizes this.

Lemma 1 (Critical Points Sufficiency)**.** *Let a_i be an AV and $\mathcal{X}_i = \{(p_j, a_i, s, e) \in \mathcal{X}\}$ be the assignments to a_i. Define the critical set $\mathcal{C}_i = \{e_i^a, x_i^a\} \cup \{s, e : (p_j, a_i, s, e) \in \mathcal{X}_i\}$. Then the point-wise capacity constraint* (3) *holds for all $x \in [e_i^a, x_i^a]$ if and only if it holds for all $x \in \mathcal{C}_i$.*

Proof. The number of PVs towed by AV a_i is a step function over $x \in [e_i^a, x_i^a]$, incrementing at segment start points s and decrementing at segment end points e. Between consecutive critical points, the count is constant. Thus, capacity satisfaction at all critical points implies satisfaction everywhere. □

This lemma is applied in both of our algorithms (Algorithms 1 and 2): instead of verifying the capacity constraint at every continuous point along an AV's route, each algorithm checks capacity only at the finite set of coupling and decoupling endpoints, reducing the feasibility check from a continuous verification to a discrete one.

F. Assumptions and Temporal Constraints

We require temporal synchronization at a coupling point cp; that is, coupling is feasible only when the following condition holds:

$$|t_i^a(cp) - t_j^p(cp)| \le \tau \tag{6}$$

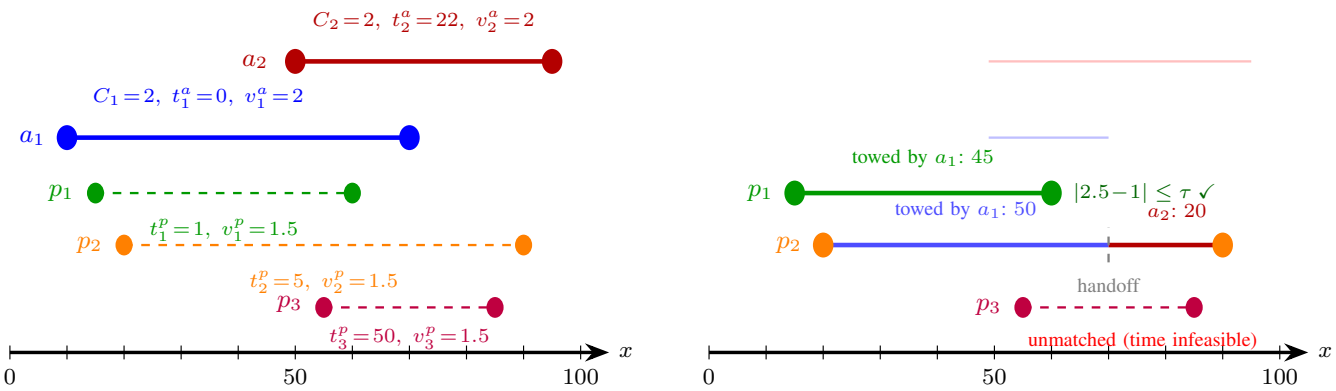
where $t_i^a(cp) = t_i^a + (cp - e_i^a)/v_i^a$ is when AV a_i reaches the coupling point, and $t_j^p(cp)$ is when PV p_j reaches the same point under its current planned trajectory. In our experiments, we use $\tau = 5$ time units with vehicle speeds drawn uniformly from $[0.8, 1.2]$ (i.e., $\pm 20\%$ around a unit reference speed). Constraint (6) captures temporal feasibility: even with spatial overlap, coupling is invalid if the two vehicles cannot meet within tolerance.

Assumption (Coupled Speed): During an attached segment, the PV follows the AV speed. Decoupling occurs when the shared segment ends or when either vehicle exits its route.

Overall, the formulation is defined by: decision variables (selected segment assignments), objective (2), and feasibility constraints including overlap, point-wise capacity, non-overlap, minimum segment length, and temporal synchronization.

G. Illustrative Example

Figure 2 illustrates a concrete instance with 2 AVs and 3 PVs on a highway of length $L = 100$. Crucially, each vehicle has an entry time and speed, and the time constraint (6) with $\tau = 5$ determines which pairings are feasible. This example demonstrates that spatial overlap alone is insufficient—temporal synchronization at the coupling point is equally decisive.



(a) Problem instance: vehicle routes with parameters.

(b) Matching result with multi-segment towing. Solid segments denote towed portions, dashed segments denote uncovered self-driving portions, and total towed distances are $p_1 = 45$, $p_2 = 70$, and $p_3 = 0$.

Fig. 2: Illustrative example ($L = 100$, $\tau = 5$) with 2 AVs and 3 PVs. (a) Problem setup with temporal parameters. (b) Matching result: multi-segment matching yields p_2 coverage of 70 units (vs. 50 with single-AV), while p_3 remains unmatched because $|t_i^a(cp) - t_3^p(cp)| > \tau$ for both AVs.

Example Analysis: With $\tau = 5$, pairings (p_1, a_1) and (p_2, a_1) are time-feasible, while p_3 is infeasible with both AVs and remains unmatched. Multi-segment matching allows p_2 to continue from a_1 to a_2 at the handoff point, increasing its covered distance from 50 to 70. The resulting total saving is 115 out of 145 baseline distance (79.3%).

IV. Proposed Algorithms

The dynamic platoon formation problem (2)–(5) generalizes the Generalized Assignment Problem (GAP), which is NP-hard [11]: each PV can be viewed as a job, each AV route $[e_i^a, x_i^a]$ as a resource bin, and the point-wise capacity constraint (3) as a bin-capacity limit. Because our formulation additionally involves multi-segment assignments, temporal synchronization, and position-dependent capacity, the problem is at least as hard as GAP. Exact solutions are therefore intractable for realistic fleet sizes. We propose two approximate algorithms: a greedy matching algorithm (Section IV-A) and an efficient Iterative Linear Assignment algorithm (Section IV-B).

A. Greedy Maximum-Weight Matching

The greedy algorithm iteratively selects the single feasible AV–PV assignment with the highest towing distance, commits to it, updates vehicle states, and repeats. Algorithm 1 operates in three phases:

Phase 1 (Initialization): For each PV p_j, initialize state $(\ell_j, \hat{t}_j) \leftarrow (e_j^p, t_j^p)$. For each AV, track current assignments for capacity checking.

Phase 2 (Candidate Generation): Enumerate all feasible (AV, PV, segment) tuples where: (1) overlap with uncovered interval $[\ell_j, x_j^p] \geq L_{min}$, (2) AV has remaining capacity at all critical points per Lemma 1, (3) time constraint (6) is satisfied.

Phase 3 (Greedy Selection): Select maximum-saving candidate, update PV and AV states, repeat until no feasible candidates remain.

Algorithm 1 Greedy Multi-AV Platoon Matching

Require: AVs $\mathcal{A}$, PVs $\mathcal{P}$, minimum length L_{min}
Ensure: Assignments $\mathcal{X}$, total saving

```
Initialize (ℓ_j, t̂_j) ← (e_j^p, t_j^p) ∀j
Initialize assignments[i] ← ∅ ∀i
X ← ∅, total ← 0
repeat
    candidates ← all feasible (AV, PV, segment) tuples
    if candidates = ∅ then
        break
    end if
    Sort candidates by saving (descending)
    (saving, a_i, p_j, cp, dp) ← candidates[0]
    X ← X ∪ {(p_j, a_i, cp, dp)}
    Update (ℓ_j, t̂_j) via (9)–(10)
    Update assignments[i]
    total ← total + saving
until no candidates
return X, total
```

Complexity. Each iteration produces $O(NM)$ candidates and sorts them in $O(NM \log(NM))$. Since greedy commits one assignment per iteration and the number of accepted segments is at most $O(NM)$, the total worst-case complexity is $O((NM)^2 \log(NM))$.

B. Iterative Linear Assignment (ILA) Algorithm

A single assignment solve is insufficient for multi-segment matching because accepting one towing segment changes the PV's residual interval and arrival time, which in turn affects the feasibility and benefit of subsequent pairings. To address this dependency across decisions, ILA solves a sequence of bipartite matching subproblems. Each iteration constructs a weighted bipartite graph from current PV states, solves the

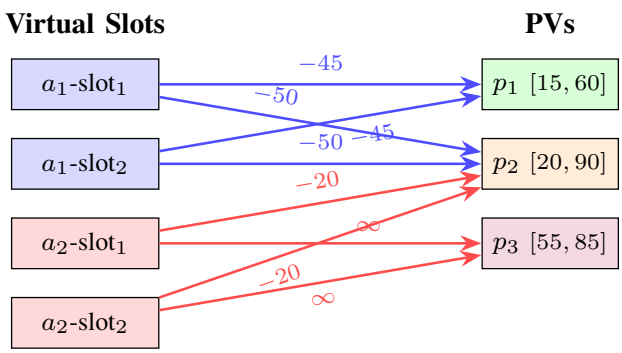


Fig. 3: Bipartite graph for ILA (iteration 1 of the toy example). Each AV with $C_i = 2$ yields two virtual slots. Edge weights are negated towed distances (∞ denotes time-infeasible pairs). The LSAP solver finds the minimum-cost matching on this graph.

resulting Linear Sum Assignment Problem (LSAP) optimally via the Jonker–Volgenant method [9], [10], [18], updates PV states to reflect newly covered segments, and repeats until no further feasible assignments exist. AV capacity is encoded through *virtual slot expansion*: each AV a_i with capacity C_i is replicated into C_i identical slots in the bipartite graph.

1) Virtual Slot Expansion and Bipartite Graph: To encode point-wise capacity, each AV a_i is expanded into C_i virtual slots, producing $K = \sum_{i=1}^{N} C_i$ left-side nodes. The M PVs form the right-side nodes. For the toy example (Section III-G), a_1 ($C_1 = 2$) yields two slots and a_2 ($C_2 = 1$) one slot ($K = 3$ left nodes); edge weights equal the overlap—e.g., a_1-p_1 has weight $\min(8, 7) - \max(2, 1) = 5$. The LSAP solver selects the maximum-weight matching: a_1-slot$_1 \to p_1$ (5) and $a_2 \to p_2$ (3). Figure 3 illustrates this.

2) Cost Matrix Construction: The cost matrix $\mathbf{W} \in \mathbb{R}^{K \times M}$ encodes the bipartite graph. For each virtual slot k belonging to AV a_i and each PV p_j:

$$W_{kj} = \begin{cases} -d_{ij}^{curr} & \text{if feasible (overlap } \geq L_{min}\text{, time ok)} \\ +\infty & \text{otherwise} \end{cases} \tag{7}$$

where $d_{ij}^{curr} = \max(0, \min(x_i^a, x_j^p) - \max(e_i^a, \ell_j))$ is the overlap between AV a_i's route and PV p_j's current uncovered interval $[\ell_j, x_j^p]$. The LSAP solver finds a minimum-cost one-to-one matching—equivalently, a matching that *maximizes* total towed distance for the current cost matrix.

3) State Update for Multi-Segment Iterations: Each PV p_j maintains state $(\ell_j, \hat{t}_j)$: ℓ_j is the *left boundary* of its still-uncovered route (initially $\ell_j = e_j^p$), and $\hat{t}_j$ is its arrival time at ℓ_j. The PV arrival time at a candidate coupling point cp is:

$$t_j^p(cp) = \hat{t}_j + \frac{cp - \ell_j}{v_j^p} \tag{8}$$

After assigning PV p_j to AV a_i on $[cp, dp]$, the state updates as:

$$\ell_j^{new} \leftarrow dp \tag{9}$$

$$\hat{t}_j^{new} \leftarrow \hat{t}_j^{old} + \frac{cp - \ell_j^{old}}{v_j^p} + \frac{dp - cp}{v_i^a} \tag{10}$$

where the first term in (10) is independent driving to the coupling point and the second is towing at AV speed.

4) Algorithm: Algorithm 2 presents the full ILA procedure.

Algorithm 2 Iterative Linear Assignment (ILA) for Multi-AV Matching

Require: AVs $\mathcal{A}$, PVs $\mathcal{P}$, minimum length L_{min}
Ensure: Assignments $\mathcal{X}$, total saving

1: Initialize states as in Algorithm 1
2: **repeat**
3: Build cost matrix $\mathbf{W}$ from current states
4: Expand AVs into $\sum_i C_i$ virtual slots
5: **if** no feasible assignments **then**
6: **break**
7: **end if**
8: $matching \leftarrow$ `SolveLSAP`($\mathbf{W}$)
9: **for** each $(slot, j) \in matching$ with $W_{slot,j} < +\infty$ **do**
10: **if** capacity feasible at all critical points (Lemma 1) **then**
11: $\mathcal{X} \leftarrow \mathcal{X} \cup \{(p_j, a_i, cp, dp)\}$; $total \mathrel{+}= (dp - cp)$
12: Update $(\ell_j, \hat{t}_j)$ via (9)–(10)
13: **else**
14: Discard; p_j remains eligible for next iteration
15: **end if**
16: **end for**
17: **until** no new assignments
18: **return** $\mathcal{X}$, $total$

Walkthrough. Line 1 initializes all PV states identically to the greedy algorithm. Lines 3–4 construct the bipartite graph: the cost matrix $\mathbf{W}$ is built from current uncovered PV intervals, and each AV is expanded into its virtual slots. Line 8 solves the LSAP on $\mathbf{W}$, returning an optimal one-to-one slot-to-PV matching for the current iteration. Lines 9–15 process each matched pair *sequentially*: for each pair, we verify point-wise capacity feasibility at all critical points (Lemma 1) given assignments already accepted *within this iteration*. Feasible pairs are accepted (with saving accumulated) and PV states are updated; infeasible pairs (arising from capacity conflicts when multiple slots of the same AV are matched) are discarded, and those PVs remain eligible in subsequent iterations. The outer loop (line 2) repeats until no new feasible assignments are found, enabling multi-segment coverage across iterations.

5) Complexity: Let $K = \sum_{i=1}^{N} C_i$ denote the total virtual slots. Each iteration constructs a $K \times M$ cost matrix in $O(KM)$ time and solves the LSAP in $O(\max(K, M)^3)$ worst-case time. Since each accepted assignment advances at least one PV boundary, the iteration count is at most $O(NM)$, giving total worst-case complexity $O(NM \cdot \max(K, M)^3)$.

C. Theoretical Analysis

The full multi-segment problem (2)–(5) is NP-hard, as it generalizes GAP [11]. We therefore analyze the guarantees of our approximate algorithms.

Theorem 2 (Per-Iteration Optimality)**.** *At each iteration of Algorithm 2, the LSAP solver returns a matching that maximizes total towed distance among all one-to-one slot-to-PV assignments for the cost matrix* $\mathbf{W}$ *constructed from current states.*

Proof. $\mathbf{W}$ encodes negated towed distances for feasible pairs and $+\infty$ otherwise. The LSAP solver finds a minimum-cost matching, which equivalently maximizes total towed distance. The Jonker–Volgenant algorithm solves LSAP exactly [18]. Convergence follows because each accepted assignment strictly advances at least one PV's boundary ℓ_j, bounding the total iterations. □

Lemma 3 (Global Optimality Gap)**.** *Algorithm 2 does not guarantee global optimality for the full multi-segment problem* (2)–(5)*. Assignments accepted in iteration* r *fix PV states for all subsequent iterations; alternative choices in round* r *could yield higher total savings globally.*

Proof. Consider a_1 overlapping p_1 on $[0, 30]$ and a_2 on $[10, 50]$. ILA may assign $p_1 \to a_1$ (saving 30), advancing ℓ_1 to 30; the remaining a_2 overlap is $[30, 50]$ (saving 20), for total 50. Yet assigning $p_1 \to a_2$ directly yields 40, freeing a_1 for other PVs—potentially producing a higher global total. □

Despite this gap, ILA consistently matches or exceeds greedy performance in all our experiments (Section V). The iterative decomposition reduces the NP-hard multi-segment problem to a polynomial sequence of tractable LSAP subproblems.

V. Experimental Evaluation

A. Experimental Setup

1) Implementation: Both algorithms are implemented in Python 3.11 using NumPy 1.26 and SciPy 1.11 [12]. The ILA algorithm uses `scipy.optimize.linear_sum_assignment`, which implements the Jonker–Volgenant shortest augmenting path method in highly optimized compiled C code. This implementation processes all slot–PV pairs in a single batch call with efficient memory access, which—combined with the small number of ILA iterations (typically 2–5)—yields lower empirical wall-clock time than greedy despite the cubic per-iteration cost. All experiments run single-threaded on an Apple M4 Pro with 16GB unified memory under macOS 15.0.

2) Scenario Generation: We generate synthetic highway scenarios with the following parameters (Table II):

Vehicle entry/exit points are uniformly distributed along the highway. Entry times follow a Poisson process, and speeds vary $\pm 20\%$ around a baseline. All results are averaged over 4 random seeds.

TABLE II: Experimental Parameters

Parameter	Values	Default
Highway length L	50–1600	100
Number of AVs N	50–80	50
Number of PVs M	200–400	200
AV capacity C	[1,2]–[1,16]	[1,3]
Minimum distance L_{min}	10	10
Time tolerance τ	5.0	5.0
Speed variation	$\pm 20\%$	$\pm 20\%$
Random seeds	42, 43, 44, 45	–

3) Metrics:

- **Baseline Distance**: $\sum_{j=1}^{M}(x_j^p - e_j^p)$, the total PV distance without any towing.
- **Total Saving**: Sum of towed distances across all assignments, i.e., $\sum_{(p_j,a_i,s,e)\in\mathcal{X}}(e-s)$.
- **Matched Ratio**: Fraction of PVs receiving at least one assignment.
- **Saving %**: $\frac{\text{Total Saving}}{\text{Baseline Distance}} \times 100$.
- **Runtime**: Wall-clock execution time in seconds.

Baseline Distance and Saving % are evaluation metrics (not optimization constraints). Under constant per-distance propulsion, Saving % is a direct proxy for PV propulsion-energy reduction, and the normalization by Baseline Distance enables fair comparison across different scenario scales.

B. Results

1) Capacity Sweep: We vary the AV capacity range C to evaluate how the number of simultaneously towable PVs affects energy savings and runtime. Table III reports results ($N = 50$, $M = 200$, $L = 100$).

TABLE III: Performance vs. AV Capacity (averaged over 4 seeds)

	Greedy			ILA		
C	Save	%	Time(s)	Save	%	Time(s)
$[1,2]$	1946	24.5	0.23	1974	24.8	**0.02**
$[1,4]$	2709	34.0	0.32	2725	34.2	**0.04**
$[1,8]$	3556	44.7	0.43	3625	45.7	**0.08**
$[1,16]$	4091	51.4	0.45	4159	52.3	**0.16**

Key observations: ILA achieves 0.2–0.9 percentage-point higher saving rates with 2.8–11.5× lower runtime. Both algorithms scale linearly with capacity; saving % improves from 24.5% ($C = [1,2]$) to 52.3% ($C = [1,16]$).

2) Highway Length Sweep: We vary the highway length L to study how spatial sparsity (fewer overlapping routes on longer highways) affects matching quality. Table IV reports results ($N = 80$, $M = 400$, $C = [1,3]$).

Key observations: Saving % decreases with highway length due to sparser vehicle overlap (31.4% at $L = 50$ vs. 7.5% at $L = 1600$). ILA shows 5.5–26.9× lower runtime; absolute savings increase but efficiency drops for longer highways.

3) Baseline Distance Comparison: Figure 4 shows the total PV distance under three conditions: (1) *Baseline*—no towing, every PV drives independently; (2) *After Greedy*—remaining

TABLE IV: Performance vs. Highway Length (averaged over 4 seeds)

	Greedy			ILA		
L	Save	%	Time(s)	Save	%	Time(s)
50	2996	31.4	1.98	3017	31.6	**0.07**
100	4594	29.8	1.59	4656	30.7	**0.11**
200	7423	26.8	1.46	7474	27.0	**0.14**
400	10742	21.2	1.39	10784	21.2	**0.15**
800	13673	13.9	0.93	13715	13.9	**0.11**
1600	14385	7.4	0.52	14481	7.5	**0.10**

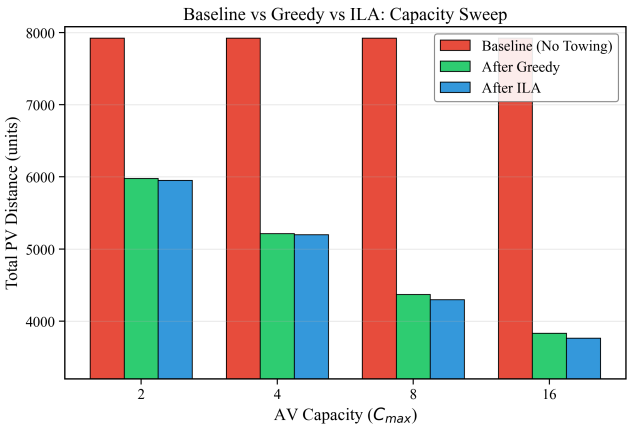


Fig. 4: Total PV distance: Baseline (no towing) vs. after Greedy vs. after ILA across AV capacity values. The difference between Baseline and each algorithm represents total distance saved (proportional to PV propulsion energy eliminated).

distance after greedy matching; (3) *After ILA*—remaining distance after ILA matching. The gap between Baseline and the optimized bars directly visualizes the energy eliminated by towing. At $C = [1, 16]$, ILA reduces total PV self-driven distance by over 52%; both algorithms achieve near-identical savings, with ILA consistently outperforming Greedy by a small margin due to its globally coordinated matching.

4) Algorithm Comparison Visualization: Figures 5 and 6 show saving percentage comparisons graphically.

5) Scalability and Runtime Observation: Figures 7 and 8 report runtime comparisons across scenarios as empirical implementation observations.

C. Discussion

Methodological Implications: The key value of ILA is structural rather than speed claims: each iteration solves its assignment subproblem optimally, virtual-slot expansion encodes AV capacity directly, and the iterative state update supports multi-segment matching while preserving feasibility constraints.

Runtime Observation (Empirical): In our Python/SciPy implementation, ILA shows lower wall-clock runtime than greedy in tested settings. This is implementation-dependent rather than a theoretical guarantee; likely contributors are SciPy's optimized C code, greedy's repeated sorting overhead, and ILA's batch processing.

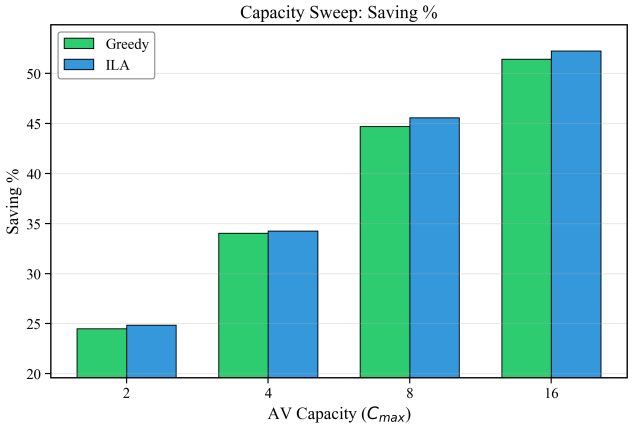


Fig. 5: Distance saving percentage vs. AV capacity for both algorithms. ILA consistently matches or exceeds greedy in solution quality.

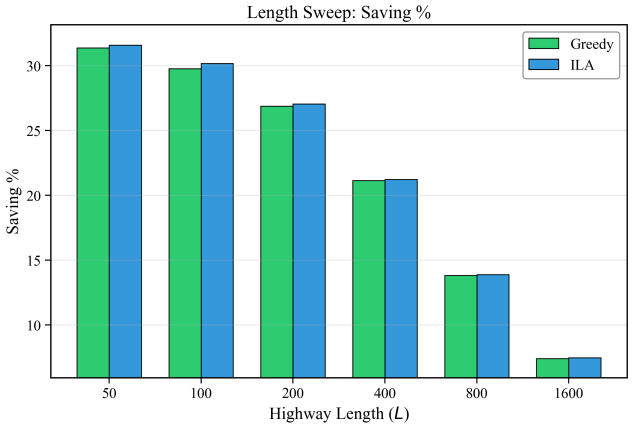


Fig. 6: Distance saving percentage vs. highway length. Both algorithms show decreasing efficiency as highways lengthen due to sparser vehicle overlap.

ILA vs. Greedy Similarity: The small saving gap (0.2–0.9 percentage points in the capacity sweep) between ILA and Greedy is expected in our one-dimensional setting, where the linear route structure limits the combinatorial diversity of feasible matchings. In higher-dimensional road networks with richer overlap patterns and branching routes, greedy's myopic decisions are more likely to block globally superior assignments, and we expect ILA's coordinated matching to yield a larger advantage.

Capacity Impact: Doubling capacity yields sub-linear improvement (24.8% $\rightarrow$ 52.3%), suggesting diminishing returns.

Length Effect: Saving % drops from 31.6% ($L = 50$) to 7.5% ($L = 1600$) due to sparser overlap, suggesting our approach is most effective for medium-length corridors with high vehicle density.

D. Limitations

Our results should be interpreted within four scope boundaries: (1) a one-dimensional highway abstraction rather than a

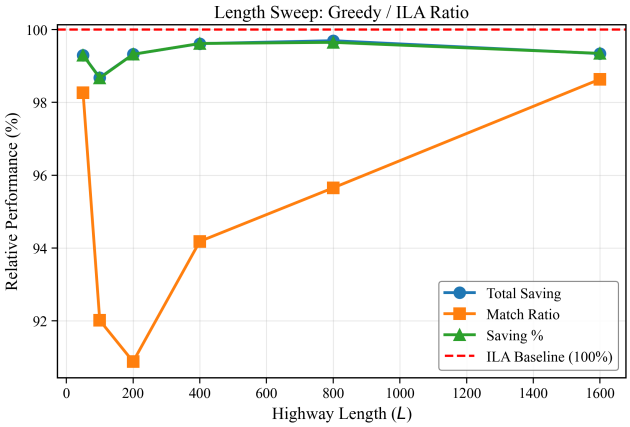


Fig. 7: Relative runtime (ILA/Greedy) across highway lengths. Values <1 indicate lower runtime for ILA in this implementation.

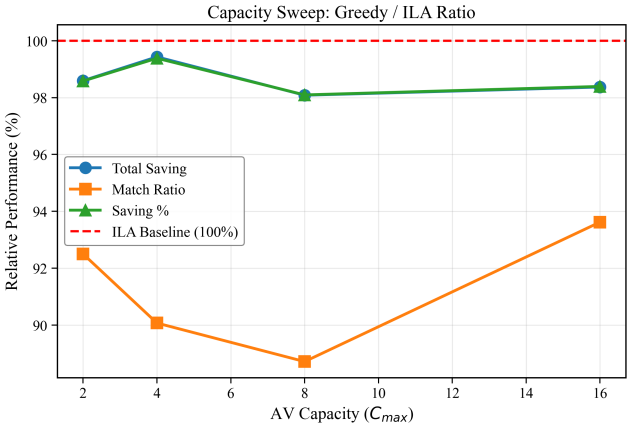


Fig. 8: Relative runtime (ILA/Greedy) across capacity values. ILA shows lower runtime in this implementation across all tested capacities.

full road network, (2) a distance-based saving proxy instead of a physics-level power model, (3) per-iteration optimality for ILA without global optimality guarantees for the full multi-segment objective, and (4) an offline setting with complete trajectory knowledge. These assumptions keep the analysis tractable and define the applicability of the reported performance.

VI. Conclusion

We presented a dynamic towing-based platoon formation framework for heterogeneous autonomous vehicles and formulated it as a constrained multi-segment matching problem on a one-dimensional corridor. Across scenarios with up to 80 AVs and 400 PVs, both algorithms achieved substantial coverage gains (25–52%), while ILA consistently matched or exceeded greedy solution quality. The framework captures point-wise capacity and temporal synchronization constraints, and the theoretical scope is explicit: ILA guarantees per-iteration optimality for each LSAP subproblem but not global optimality for the full multi-segment objective.

A. Future Work

Future work will extend the model to two-dimensional road networks, evaluate online and uncertain traffic settings with periodic re-optimization, and investigate decentralized coordination and incentive mechanisms for large-scale deployment. We will also validate the distance-based saving proxy against physics-based energy models and practical coupling constraints using real-world trajectory datasets and AV/PV platform studies.

References

[1] U.S. Department of Transportation, Federal Highway Administration, "Summary of Travel Trends: 2022 National Household Travel Survey," Report FHWA-HPL-24-009, 2024.

[2] U.S. Environmental Protection Agency, "Inventory of U.S. Greenhouse Gas Emissions and Sinks: 1990–2021," EPA 430-R-23-003, 2023.

[3] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A survey of autonomous driving: Common practices and emerging technologies," *IEEE Access*, vol. 8, pp. 58443–58469, 2020.

[4] S. Tsugawa, S. Jeschke, and S. E. Shladover, "A Review of Truck Platooning Projects for Energy Savings," *IEEE Trans. Intell. Veh.*, vol. 1, no. 1, pp. 68–77, 2016.

[5] C. Bergenhem, H. Pettersson, E. Coelingh, C. Englund, S. Shladover, and S. Tsugawa, "Overview of platooning systems," in *Proc. 19th ITS World Congress*, Vienna, Austria, 2012.

[6] M. Furuhata, M. Dessouky, F. Ordóñez, M.-E. Brunet, X. Wang, and S. Koenig, "Ridesharing: The state-of-the-art and future directions," *Transport. Res. Part B: Methodol.*, vol. 57, pp. 28–46, 2013.

[7] N. Agatz, A. Erera, M. Savelsbergh, and X. Wang, "Optimization for dynamic ride-sharing: A review," *European J. Oper. Res.*, vol. 223, no. 2, pp. 295–303, 2012.

[8] J.-F. Cordeau and G. Laporte, "The dial-a-ride problem: models and algorithms," *Annals Oper. Res.*, vol. 153, no. 1, pp. 29–46, 2007.

[9] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[10] J. Munkres, "Algorithms for the assignment and transportation problems," *J. Soc. Ind. Appl. Math.*, vol. 5, no. 1, pp. 32–38, 1957.

[11] D. G. Cattrysse and L. N. Van Wassenhove, "A survey of algorithms for the generalized assignment problem," *European J. Oper. Res.*, vol. 60, no. 3, pp. 260–272, 1992.

[12] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[13] S. E. Shladover, "Cooperative (rather than autonomous) vehicle-highway automation systems," *IEEE Intell. Transp. Syst. Mag.*, vol. 1, no. 1, pp. 10–19, 2009.

[14] J. Ploeg, B. T. M. Scheepers, E. van Nunen, N. van de Wouw, and H. Nijmeijer, "Design and experimental evaluation of cooperative adaptive cruise control," in *Proc. IEEE Int. Conf. Intell. Transp. Syst.*, pp. 260–265, 2011.

[15] M. Drexl, "Synchronization in vehicle routing—a survey of VRPs with multiple synchronization constraints," *Transport. Sci.*, vol. 46, no. 3, pp. 297–438, 2012.

[16] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, and D. Rus, "On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment," *Proc. Natl. Acad. Sci. U.S.A.*, vol. 114, no. 3, pp. 462–467, 2017.

[17] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *J. Artif. Intell. Res.*, vol. 31, pp. 591–656, 2008.

[18] R. Burkard, M. Dell'Amico, and S. Martello, *Assignment Problems*, SIAM, 2012.